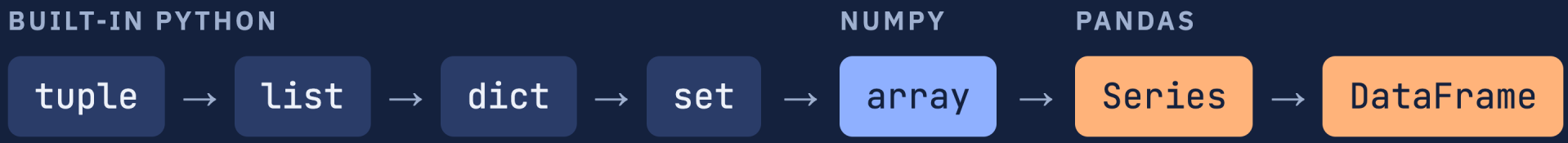
PYTHON FOR BEGINNERS

From tuple to DataFrame

Seven containers. Each one adds a single idea to the one before.



Built-in containers: everyday pictures

`tuple`

A date of birth

Day, month, year: written once, always in the same order. You read it, you never edit it.

You get safety: nobody changes it by accident.

`list`

A shopping list

You add items, cross them out, swap them. Each item has a place: first, second, third.

You get flexibility: it changes as you work.

`dict`

A phone book

You don't remember that Ann is contact number 57. You ask for "Ann" and get her number.

You get lookup by name: instant, however big the book is.

`set`

A guest list

Each person is on it once. The only question you ask: is this person on the list?

You get uniqueness: duplicates disappear by themselves.

Built-in containers: one change per step

START HERE

`tuple`

Ordered and fixed: once created, it cannot change.

```
p = (55.75, 37.62)
p[0] # 55.75
p[0] = 1 # error
```

Use for a record that must stay as it is: coordinates, (name, age).

+ ALLOW CHANGES

`list`

Still ordered, but you can add, remove and replace items.

```
xs = [3, 1, 2]
xs.append(5)
xs[0] = 10 # fine
```

Use for a sequence that grows or changes: lines of a file, results of a loop.

+ ACCESS BY KEY

`dict`

You find a value by its key, not by its position.

```
age = {"Ann": 23}
age["Bob"] = 25
age["Ann"] # 23
```

Use for lookup by name: settings, word counts, id → record.

– DROP THE VALUES

`set`

A dict with only the keys left: unique items, no order.

```
s = {"a", "b", "a"}
len(s) # 2
"a" in s # True
```

Use for uniqueness and fast "is it in?" checks: removing duplicates.

NumPy array: a list of lists with rules

RULE 1

One type

Every element has the same type, usually a number.

RULE 2

Rectangular shape

Rows × columns: every row has the same length.

WHAT YOU GET FOR IT

Math without loops

One operation applies to all elements at once, and it is fast.

```
rows = [[1, 2, 3], [4, 5, 6]]   a list of lists
a = np.array(rows)              now an array
a.shape                         (2, 3): 2 rows, 3 columns
a * 10                          every element × 10
a[:, 0]                         first column: [1, 4]
rows * 2                        a list only repeats itself
```

Use for numbers of one type and math on all of them: vectors, matrices, images, simulations.

pandas: an array with labels

A VECTOR

1-D array

position	values
0	23
1	25
2	31

Values are found by position only.

+ AN INDEX

Series

index	values
Ann	23
Bob	25
Eva	31

Each value gets a label, so s["Ann"] gives 23, like a dict.

+ MORE SERIES, SIDE BY SIDE

DataFrame

age	city	paid	Ann
23	Moscow	True	Bob
25	Kazan	False	Eva
31	Omsk	True	

Columns share one index. Each column keeps its own type: int, str, bool.

DataFrame = NumPy array + own type per column + row labels (index) + column labels (columns)

pandas: why labels matter

INTUITION 1

Labels keep the right numbers together

```
jan = pd.Series({"Ann": 10, "Bob": 20})
feb = pd.Series({"Bob": 5, "Ann": 1})
jan + feb # Ann 11, Bob 25
np.array([10, 20]) + np.array([5, 1])
# [15, 21]: Ann got Bob's number
```

Arrays match values by position. pandas matches them by label, so the order of the rows stops being a source of errors.

INTUITION 2

A DataFrame is a dict of columns

```
ages = [23, 25, 31]
cities = ["Moscow", "Kazan", "Omsk"]
d = {"age": ages, "city": cities}
df = pd.DataFrame(d)
df["age"] # one column, a Series
```

The built-ins come back: lists become columns, dict keys become column names, and df["age"] reads like a dict lookup.

The path: every step is a trade

Step	You give up	You get
tuple → list	protection from accidental change	freedom to add, remove and replace
list → dict	access by position	instant lookup by a meaningful key
dict → set	the values	uniqueness and fast "is it in?" checks
list of lists → array	mixed types, rows of different lengths	speed and math on everything at once
array → Series	a little speed and simplicity	labels: lookup and alignment by name
Series → DataFrame	one type for all the data	a table whose columns can differ in type

No container is the best one. Each is the best at one job, and that is why Python keeps all seven.

Which container to choose

Container	Choose it when	Typical example
tuple	A small fixed record should never change	(latitude, longitude)
list	An ordered collection grows or changes	lines of a file, results of a loop
dict	You look things up by name, not by position	word → count, id → customer
set	You need unique items or fast "is it in?" checks	distinct cities, ids already seen
NumPy array	Numbers of one type, math on all of them at once	matrices, images, simulations
Series	One column of data with labels	daily prices indexed by date
DataFrame	A table: columns of different types, labelled rows	a CSV file, an Excel sheet, an SQL result

Rule of thumb: a table from a file goes into a DataFrame, pure numeric math into a NumPy array, and everything small in between into a list or a dict.